

The "Big 5" Characters to Escape (XSS Prevention)

Why HTML Entity Encoding Matters

Cross-Site Scripting (XSS) is one of the most common web vulnerabilities. Attackers inject malicious scripts into your pages that execute in other users' browsers.

The Solution: Convert dangerous characters into HTML entities before rendering user input.

The Big 5 Characters

To prevent XSS, you **MUST** convert these five characters into HTML entities:

Character	Entity Name	Entity Number	Why Dangerous
&	&	&	Starts entity sequences
<	<	<	Opens HTML tags
>	>	>	Closes HTML tags
"	"	"	Breaks attribute quotes
'	'	'	Breaks attribute quotes (single)

Attack Examples

Attack 1: Script Injection

HTML

```
<!-- User input: <script>alert('XSS')</script> -->
```

❌ VULNERABLE (not escaped):

```
<div>User said: <script>alert('XSS')</script></div>
<!-- Script executes! -->
```

✅ SAFE (escaped):

```
<div>User said: &lt;script&gt;alert('XSS')&lt;/script&gt;</div>
<!-- Displays as text: "<script>alert('XSS')</script>" -->
```

Attack 2: Image onerror

HTML

```
<!-- User input: <img src=x onerror="alert('XSS')"> -->
```

❌ VULNERABLE:

```
<div><img src=x onerror="alert('XSS')"></div>
```

✅ SAFE:

```
<div>&lt;img src=x onerror="alert('XSS')"&gt;</div>
```

Attack 3: Attribute Breaking

HTML

```
<!-- User input: " onmouseover="alert('XSS') -->
```

❌ VULNERABLE:

```
<input type="text" value="" onmouseover="alert('XSS')">
<!-- Breaks out of value attribute! -->
```

✅ SAFE:

```
<input type="text" value="&quot; onmouseover=&quot;alert('XSS')">
<!-- Stays within value attribute -->
```

Implementation

JavaScript (Browser)

JavaScript

```
function escapeHtml(unsafe) {
    return unsafe
```

```

        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#x27;");
    }

    const userInput = '<script>alert("XSS")</script>';
    const safe = escapeHtml(userInput);
    document.getElementById('output').textContent = safe;
    // Displays: &lt;script&gt;alert(&quot;XSS&quot;)&lt;/script&gt;

```

JavaScript (Modern - using textContent)

```

JavaScript
// Best practice: Use textContent (automatically escapes)
const userInput = '<script>alert("XSS")</script>';

// ❌ VULNERABLE:
element.innerHTML = userInput; // Script executes!

// ✅ SAFE:
element.textContent = userInput; // Automatically escaped

```

React (Automatic Escaping)

```

None
// React automatically escapes by default
function Comment({ text }) {
    return <div>{text}</div>; // Safe! React escapes automatically
}

// ⚠️ DANGER: dangerouslySetInnerHTML bypasses escaping
function UnsafeComment({ html }) {
    return <div dangerouslySetInnerHTML={{ __html: html }} />;
    // Only use if HTML is sanitized!
}

```

PHP

```

PHP
// htmlspecialchars is your friend

```

```
$userInput = '<script>alert("XSS")</script>';

// ✅ SAFE:
echo htmlspecialchars($userInput, ENT_QUOTES, 'UTF-8');
// Output: &lt;script&gt;alert(&quot;XSS&quot;)&lt;/script&gt;

// Or use htmlentities for more comprehensive encoding
echo htmlentities($userInput, ENT_QUOTES, 'UTF-8');
```

Python (Flask/Jinja2)

```
Python
from markupsafe import escape

user_input = '<script>alert("XSS")</script>'

# ✅ SAFE (manual):
safe_output = escape(user_input)
# → &lt;script&gt;alert(&#34;XSS&#34;)&lt;/script&gt;

# Jinja2 templates auto-escape by default:
# {{ user_input }} → Automatically escaped
# {{ user_input|safe }} → NOT escaped (dangerous!)
```

Python (Django)

```
Python
from django.utils.html import escape

user_input = '<script>alert("XSS")</script>'
safe_output = escape(user_input)

# Django templates auto-escape by default:
# {{ user_input }} → Automatically escaped
# {{ user_input|safe }} → NOT escaped (dangerous!)
```

Common Mistakes

✗ Mistake 1: Only Escaping Some Characters

```
JavaScript
// ✗ WRONG: Only escapes <
function badEscape(str) {
    return str.replace(/</g, "&lt;");
}

// Attack still works:
badEscape('"><script>alert("XSS")</script>');
// → "><script>alert("XSS")&lt;/script>"
// Double quote breaks attribute, script still executes!
```

✗ Mistake 2: Escaping in Wrong Order

```
JavaScript
// ✗ WRONG ORDER:
function wrongOrder(str) {
    return str
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/&/g, "&amp;"); // This will escape the & in &lt; and &gt;!
}

wrongOrder("<div>");
// → "&amp;lt;div&amp;gt;" (double-escaped!)

// ✓ CORRECT: Always escape & first
function correctOrder(str) {
    return str
        .replace(/&/g, "&amp;") // First!
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;");
}
```

✗ Mistake 3: Using innerHTML Instead of textContent

```
JavaScript
// ✗ WRONG:
element.innerHTML = escapeHtml(userInput);
// Still vulnerable if escapeHtml has bugs!

// ✓ BETTER:
```

```
element.textContent = userInput;  
// Browser handles escaping
```

Context-Specific Encoding

Different contexts require different escaping:

HTML Content

```
HTML  
<div>{{ ESCAPE_HTML(userInput) }}</div>
```

HTML Attributes

```
HTML  
<input value="{{ ESCAPE_HTML(userInput) }}">  

```

<!-- Need CSS escaping -->

Complete HTML Entities Reference

Common Special Characters

Character	Name	Entity Name	Entity Number
&	Ampersand	&	&
<	Less Than	<	<
>	Greater Than	>	>
"	Double Quote	"	"
'	Single Quote	'	' or '
	Space	 	
©	Copyright	©	©
®	Registered	®	®
™	Trademark	™	™
€	Euro	€	€
£	Pound	£	£
¥	Yen	¥	¥
•	Bullet	•	•
←	Left Arrow	←	←
→	Right Arrow	→	→
↑	Up Arrow	↑	↑
↓	Down Arrow	↓	↓

Content Security Policy (CSP)

Additional layer of protection against XSS:

HTML

```
<!-- Add to <head> or HTTP header -->
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self'; script-src 'self'; style-src 'self';">
```

What it does:

- Blocks inline scripts (`<script>alert('XSS')</script>`)
- Only allows scripts from your domain
- Prevents eval() and inline event handlers

Example Policy:

None

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' https://cdn.trusted.com;
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https:;
  font-src 'self' https://fonts.gstatic.com;
```

Sanitization Libraries

For complex HTML (WYSIWYG editors, Markdown):

JavaScript: DOMPurify

JavaScript

```
import DOMPurify from 'dompurify';

const dirty = '<img src=x onerror=alert("XSS")>';
const clean = DOMPurify.sanitize(dirty);
// → '' (onerror removed)

// Allow specific tags
const clean = DOMPurify.sanitize(dirty, {
  ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a'],
```

```
    ALLOWED_ATTR: ['href']
});
```

Python: Bleach

```
Python
import bleach

dirty = '<script>alert("XSS")</script><b>Bold</b>'

clean = bleach.clean(
    dirty,
    tags=['b', 'i', 'em', 'strong', 'a'],
    attributes={'a': ['href', 'title']},
    strip=True
)
# → '<b>Bold</b>'
```

PHP: HTML Purifier

```
PHP
$config = HTMLPurifier_Config::createDefault();
$purifier = new HTMLPurifier($config);

$dirty = '<script>alert("XSS")</script><b>Bold</b>';
$clean = $purifier->purify($dirty);
// → '<b>Bold</b>'
```

Testing for XSS

Manual Testing Payloads

```
HTML
<script>alert('XSS')</script>
<img src=x onerror=alert('XSS')>
<svg/onload=alert('XSS')>
"><script>alert('XSS')</script>
'><script>alert('XSS')</script>
<iframe src="javascript:alert('XSS')">
```

```
<body onload=alert('XSS')>
<input onfocus=alert('XSS') autofocus>
```

Automated Testing

```
JavaScript
// Jest/Vitest example
describe('XSS Protection', () => {
  const xssPayloads = [
    '<script>alert("XSS")</script>',
    '<img src=x onerror=alert("XSS")>',
    '"><script>alert("XSS")</script>',
    '<svg/onload=alert("XSS")>'
  ];

  xssPayloads.forEach(payload => {
    it(`escapes payload: ${payload}`, () => {
      const escaped = escapeHtml(payload);
      expect(escaped).not.toContain('<script');
      expect(escaped).not.toContain('onerror=');
      expect(escaped).toContain('&lt;');
    });
  });
});
```

Security Checklist

- ☐ Escape all user input before displaying
 - ☐ Use `textContent` instead of `innerHTML` when possible
 - ☐ Never trust user input (even from your database)
 - ☐ Implement Content Security Policy (CSP)
 - ☐ Use templating engines that auto-escape
 - ☐ Sanitize rich text with approved libraries
 - ☐ Test with XSS payloads
 - ☐ Review code for `dangerouslySetInnerHTML`, `v-html`, etc.
 - ☐ Never use `eval()` with user input
 - ☐ Validate and sanitize on server-side (client-side is not enough)
-

Real-World Examples

WordPress (Escaping Output)

PHP

```
//  SAFE:  
echo esc_html($user_comment);           // For HTML content  
echo esc_attr($user_input);             // For HTML attributes  
echo esc_url($user_url);                // For URLs  
echo esc_js($user_data);                // For JavaScript
```

Ruby on Rails

None

```
<!-- Auto-escapes by default -->  
<%= user_input %>  
  
<!-- Raw output (dangerous!) -->  
<%= user_input %>  
<%= raw user_input %>
```

Angular

HTML

```
<!-- Auto-escapes by default -->  
<div>{{ userInput }}</div>  
  
<!-- Bypass sanitization (dangerous!) -->  
<div [innerHTML]="userInput"></div>
```

Need Security Help?

Secure your code?

Devxhub writes secure, enterprise-grade software with comprehensive XSS protection:

-  Security code reviews
-  Penetration testing
-  XSS/CSRF protection implementation
-  Security training for teams
-  OWASP Top 10 compliance

Our Services:

- Security audit & vulnerability assessment
- Secure coding training
- Web application firewall (WAF) setup
- Security monitoring & incident response

 Contact us: info@devxhub.com

 Visit: www.devxhub.com

 Call: +880 1326 506464

© 2026 Devxhub Private Limited. All rights reserved.